

Using OpenStack To Improve Student Experience in an H.E. Environment

Stephen Bonner, Carl Pulley, Ibad Kureshi, Violeta Holmes, John Brennan, Yvonne James
School of Computing and Engineering
University of Huddersfield
Huddersfield, UK HD1 3DH
Email: hpc-rg@hud.ac.uk

Abstract—The cloud computing paradigm promises to deliver hardware to the end user at a low cost with an easy to use interface via the internet. This paper outlines an effort at the University of Huddersfield to deploy a private Infrastructure as a Service cloud to enhance the student learning experience. The paper covers the deployment methods and configurations for OpenStack along with the security provisions that were taken to deliver computer hardware. The rationale behind the provisions of virtual hardware and OS configurations have been defined in great detail supported by examples. This paper also covers how the resource has been used within the taught courses as a Virtual Laboratory, and in the research projects. A second use case of the cloud for Automated Formative Assessment (AFA) by using JClouds and Chef for Continuous Integration is presented. The AFA deployment is an example of a Software as a Service offering that has been added on to the IaaS cloud. This development has led to an increase in freedom for the student.

I. INTRODUCTION

The term cloud computing is often applied to diverse array of internet based computing services, and providing a concrete definition is often difficult. The variety of different technologies, along with the hype surrounding cloud computing can further complicate the issue. However in a broad sense, cloud computing is designed to enable convenient, on demand access to a collection of user configurable computational resources (including storage, network and processors) that can be rapidly redeployed in a variety of different operating systems and hardware configurations [1].

As well as offering the end user benefits associated with ease of use, cloud computing offers considerable advantages to the institution offering such services. The main one being the reduction in costs associated with procuring and managing hardware and software resources by allowing them to be multipurpose and easy for the end user to self manage [2]. Using cloud computing means that less physical hardware is needed to meet user demands, which positively affects the electricity usage and staff hours needed to manage these resources.

Cloud computing offers a variety of different service models in which the resources can be deployed [3] -

- *Infrastructure as a Service (IaaS)* - This service model allows the resources to be deployed directly as fundamental computing resources including processing, storage and network resources. Using this hardware the end user can run arbitrary software including

operating systems or applications. The end user does not manage the underlying fixed infrastructure used to provide the cloud services but instead can control how the resources are deployed.

- *Software as a Service (SaaS)* - This service model allows the end user to use software provided by the same provider as the hardware. The user accesses these services via a thin client interfaces such as web browsers.
- *Platform as a Service (PaaS)* - This service model allows custom applications to be deployed into the cloud without the need for the end user to manage the hardware. Examples of this would include Microsoft's Azure or Google's App Engine.

We believe that the possibilities offered by the flexible IaaS model could be of great advantage in teaching computing modules in a higher education environment. The demands of students undertaking computer science degrees in the twenty-first century have changed [4]. This not only includes an understanding of how to configure network devices but also how to install, configure and use a variety of operating systems.

In order to deploy cloud infrastructure into a *Virtual Laboratory* there were a number of decisions required that relate to:

- 1) Creating student and staff user accounts, either individually or as a group.
- 2) Enabling access to internal and external cloud resources.
- 3) Ensuring that the Universities network security is not compromised.
- 4) Selecting software and hardware services required for curriculum delivery.

The next section of this paper covers our motivations behind developing a private cloud and how this has led to a dedicated resource for teaching and learning. Section III covers aspects of OpenStack deployment within the University of Huddersfield environment. This section touches upon security mechanisms (network and encryption) and sign up processes for student users. In Section IV we discuss two use cases: the first being the use of the cloud as a Virtual Classroom and the second as the cloud as an Automated Formative Assessment tool. In the fifth section we conclude our findings and reflect on the effectiveness of this solution.

II. MOTIVATION

The High Performance Computing Research Group (HPC-RG) at the University of Huddersfield aims to be at the cutting edge of Research, Distributed and Parallel Computing technologies. To better understand the mechanisms of the cloud computing paradigm we developed a small cloud internally. The benefits of the cloud such as low cost delivery of hardware, lower provision time etc, became quickly apparent.

For researchers within the HPC-RG the private cloud served as an improved platform to test and develop cluster and grid middlewares. Within the cloud, snapshots of the existing clusters could be deployed and the effects of updates could be monitored before actual deployment. Previously this type of testing and developing was done using an old and very power inefficient cluster.

Technical support officers at the High Performance Computing Resource Centre (HPC-RC) saw cloud computing as an effective method to provision servers for small research projects. Students and researchers did not need access to sensitive machine rooms, and unlike XEN and VMWare based server farms, support officers would not need to get involved in reinstallation. The cloud instances also put a stop to the use of the older energy inefficient hardware.

As a consequence of the cloud deployment for research purposes, it was anticipated, that the student experience would be greatly increased if students were able to utilise this facility as a Virtual Laboratory. To achieve this goal a production grade cloud was deployed to meet the planned usage with the following requirements:

- 1) The system should be centralised and available to all faculty or modules for use by their students
- 2) The cloud should give the students greater freedom to explore, experiment and configure systems to their specifications
- 3) The added freedom should not compromise security of the campus network.
- 4) Accessing the virtual systems should not be more complicated or time consuming than visiting a laboratory.
- 5) Apart from initial training this service should not add more work for the faculty or the lab/resource technicians as compared to running a lab.

In the following section we have discussed the structure and layout of the system that was developed over the summer of 2012. This system remained in used in the academic year for 2012-2013 from the months of October till April.

III. UNIVERSITY OF HUDDERSFIELD DEPLOYMENT

OpenStack is quickly becoming the de-facto standard for IaaS Cloud platforms and is generating a lot of excitement in the research community [5]. OpenStack is a joint project between the National Aeronautics and Space Administration (NASA) and Rackspace that implements the IaaS platform via a collection of open-source cloud management packages [6]. Currently these packages include Nova (OpenStack's compute and network module), Glance (Image management module), Keystone (User management module) and Swift (Storage module) [7] [8]. One of the key advantages to OpenStack is its

greater flexibility in the management of users. For example all users are assigned to Projects, which can be granted accesses to a certain percentage of the computational resources [5]. Also, unlike Eucalyptus, users can register and upload their own custom ramdisk and kernel images [5].

Before deploying an OpenStack based cloud, we had previously experimented with an Ubuntu Enterprise Cloud (UEC), and a Proxmox Cloud but neither were complete satisfactory. We found UEC to have a very steep learning curve and Eucalyptus, on which UEC is based, was removed from Ubuntu 11.10 onwards. We established that Proxmox was more suited as an open-source replacement for VM Ware, rather than being a full featured IaaS platform. At the time of deployment, the stable version of OpenStack was Essex, and as such this was the version used.

For our deployment the following underlying hardware was utilised:

- The Controller Node - Intel Pentium D E5700 3.0Ghz, 2GB Ram, Ubuntu Server 12.04 LTS
- The Compute Nodes - 2 * 24 core AMD Opteron 6168, 40GB RAM, Ubuntu Server 12.04 LTS

The nodes communicate via a 1GB private switch. The controller node is configured to do all of the backend processes including, image management via Glance, hosting the website via Horizon and dealing with user identification via Keystone. The controller node is not running Nova compute, as the two more powerful machines are configured to be dedicated Nova compute elements.

The network layout for the cloud can be seen below.

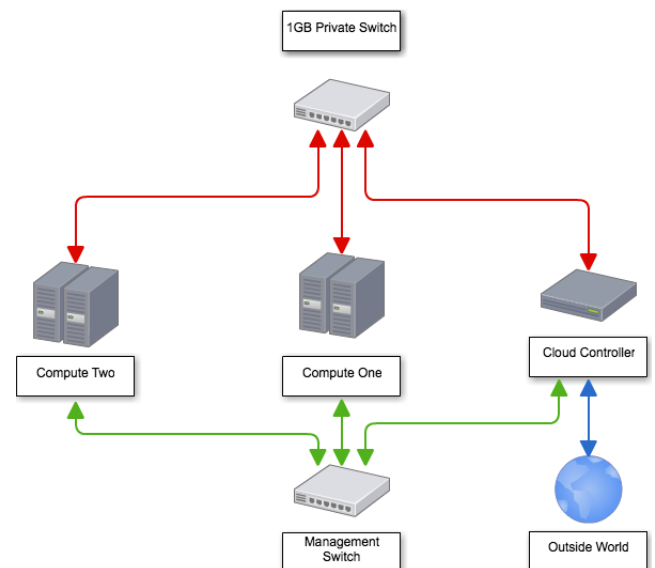


Fig. 1. Cloud Network Topology.

A. Security

1) *User Authentication:* One aspect of the OpenStack deployment that lacked features we required was the procedure for adding users. There are two methods available in the

standard software stack, a Graphical User Interface (GUI) and a Command Line Interface (CLI) [7] neither of which was really suitable for adding a batch of users. This was required when adding a group of students associated with a specific module. While the standard GUI interface was very intuitive when used to create accounts, it required a web based form to be completed by the administrator for each user. This method assigns the user a primary group allowing the account to be usable immediately. Previously this had been a time consuming method for adding multiple users and did not allow for a user role to be specified.

We also considered using the CLI directly to add users. This transpired to be quite time consuming, as a user must first be created using a user-name. In order to make the account usable it must be given a role. However this required the use of the users hash (a string of characters), which is generated in the process of adding the user. In order to add the user to a role, the system needed to be queried for the hash, so the user could then be added to a role [7]. Again this method was unsuitable for adding multiple users, as it took a great deal of time to complete and was very prone to errors.

A limitation of both standard methods to add users was that the passwords needed to be set on account creation, and could only be changed by an administrator. This left the responsibility of generating unique passwords, which did not follow a predictable pattern, on the administrator creating the account. The password needed to be communicated to the user and that again takes a great deal of time when adding multiple users.

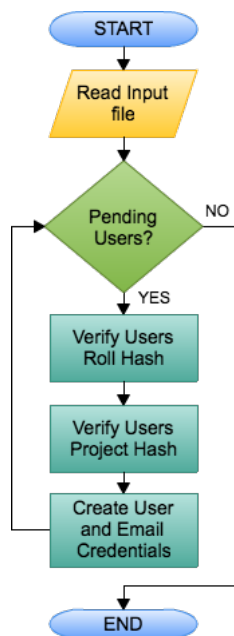


Fig. 2. Flowchart for Group Add Script

In order to overcome these problems a script was developed to automate the process for adding a batch of users (see Figure 2). This script reads user information in the form of user-name, given name, email, primary project and role. Further information such as hashes for roles and projects were hard-coded into the script, as it could be reasonably assumed that these would not change. The final piece of information required

Flavour Name	VCPUs	Memory	Root Disk	Ephemeral Disk
m2.xlarge	16	32786	20	20
m2.medium	2	4096	20	20
m2.small	1	2048	20	20
m1.xlarge	8	16384	10	160
m1.large	4	8192	10	80
m1.medium	2	4096	10	40
m1.small	1	2048	10	20

TABLE I. OFFERED HARDWARE FLAVOURS

was a unique password for each user, this was generated with the use of `/dev/urandom` [9] to be a sufficiently long alphanumeric string. With all this information the script would add the user, determine the given user hash, set the users role and then email the user with all the information required to log on the system.

2) *Floating Network*: For security reasons we wanted to control users access to the external network from within cloud instance. This was achieved by limiting user access to the bank of floating IP addresses. These floating IP addresses were assigned from the Universities VLAN. This has the advantage that the Universities firewall policies are automatically applied to these addresses.

B. Service Offerings

We offer the following operating systems from which users can spawn cloud instances -

- Ubuntu 10.04 64Bit
- Ubuntu 12.04 64Bit
- Centos 6 64Bit
- Fedora 16 64Bit
- Windows 7 64Bit
- Windows XP SP3 32Bit

These operating systems were specifically chosen to deal with the breadth of computing needs at the university. By default the Ubuntu images come with OpenStack. Fedora Core is offered so to help students contrast between DEB and RPM based operating systems. CentOS was chosen, as it is used as the main operating system in the High Performance Computing module taught at the university, and is a good server class OS. Windows 7 is the primary campus operating system and therefore is required to provide different software packages for CAD or 3D rendering. Within the Operating Systems course the Windows research kernel is studied and therefore a Windows XP image is also provided.

We offer a range of hardware 'flavours' from which the users can choose their instances. The flavours were designed to meet the diverse range of computational needs of the users.

One of the most frequent requests from our users is for high memory instances needed for 3D, meshing and CAD processing tasks (See table I). Therefore an instance was created, featuring 32GB of memory with 16 CPU cores. For teaching and experience purposes, that only require limited resources, the small instance can be utilised to allow many simultaneous users. To manage the limited hardware resources

and enable us to offer a consistent service to all users, we have placed limits on the quota of CPU's and memory that each project can utilise. For example, the project with the largest amount of users has a strict limit on the resources available, to enable fair access for users working on other projects.

IV. USE CASES

A. Virtual Laboratory

The demands of students undertaking computer science degrees in the twenty-first century have changed. While the basics have remained the same and students still need to understand computer architecture and network infrastructure, the computing industry now requires students to have more hands-on technical skills. This not only includes an understanding of how to configure network devices but also how to install, configure and use a variety of operating systems.

We have utilised the private cloud service at the University of Huddersfield to facilitate the delivery of undergraduate and postgraduate modules within the computer science degrees. This has allowed students to natively gain experience with a variety of Windows and Linux operating systems. By providing virtual systems for the students, which they could instantiate in their own time, the learning experience was greatly enhanced. Rather than using the traditional hardware box in a lab students were able to easily and quickly launch and configure many different operating systems without requiring repeated reinstallation of a physical machine (see Figure 3).

The cloud service allowed faculty members to deliver the hardware to the students via the internet. This saved on premium lab space [10] and students benefitted from being able to access the cloud both on and off campus, any time of night or day to complete labs and assignments. In this sense the cloud has provided an on-demand service to further facilitate and heighten the learning experience. Further different modules like 'Operating Systems' and 'Distributed Systems' (typically dealing with Server-Client based programming and configurations) did not need dedicated labs or hardware. Technician time was also saved as machines did not need to be re-provisioned between different classes or if a machine stopped working.

Instances

☐	Tenant	Host	Instance Name	Status	Power State
☐	Ibad's-Work	cloudcompute1	test	Active	Running
☐	CIS2380 - OSLT	cloudcompute2	Mani_Dana	Active	Running
☐	CIS2380 - OSLT	cloudcompute2	dimmy	Active	Running
☐	CIS2380 - OSLT	cloudcompute2	ubuntuserver	Active	Running
☐	CIS2380 - OSLT	cloudcompute2	First Server	Active	Running

Displaying 5 items

Fig. 3. Screenshot of Running Instances: Students of the Operating System Course can be seen with the course code Tenant ID CIS2380-OSLT.

The use of the cloud to facilitate teaching has meant that there is no longer a need to provide one PC or swappable

hard drive per student. With broader adoption this will result in a significant cost saving over time, which is currently unquantified. There is also a considerable reduction in the amount of hardware involved. This means that there is also a reduced chance of hardware failures. Any severe failure can be rectified by re-spawning the image again, cutting down time and effort.

For student projects based around LAMP (Linux, Apache, MySQL and PHP) type configurations, the cloud provided a good solution for low cost hardware provisioning. Control of the LAMP configurations were for the first time in the hands of the students. With snapshot facilities the students were also able to both secure their work and show progression for provenance.

The adopted model of education has moved away from traditional instructive teacher-led models towards a more student-centred constructivist approach [11]. To facilitate this move it was necessary to ensure that teaching staff had the skills in order to provide the required levels of instruction and support. Students are also encouraged to learn by doing and to experiment with different operating systems. This provided the opportunity to ask "What works?" and encouraged the students to adopt a trial and error approach. To allow this, the students are given root access for Linux and full administrator rights for Windows. Typically such access was restricted when students were using bare metal machines. Using the in-built security mechanisms (security groups and network rules) of OpenStack and the University Network, security of the campus network was maintained.

Second year students were given a chance to experiment with Linux and this has enhanced their skills in command line configuration and scripting. These skills are very important in more advanced computing courses such as Parallel Computing Architectures, Computer Clusters and Grids.

The private cloud service was also offered to student projects from other disciplines like Art and Design and Mechanical Engineering. Students handling complex computational problems were able to use high memory hardware flavours, within the cloud, to meet their project goals. CAD models and 3D renders that exceeded memory limits on standard desktops and were not HPC compatible, were easily completed using Windows based virtual machines. The snapshots with the CAD and 3D rendering softwares, then became a non-traditional SaaS offering. While the software is not delivered through the browser, the virtual machine with the software is.

B. Automated Formative Assessment

In this use case we look at how the HPC cloud has been used to automatically generate formative feedback to submitted student assessments. Here we focus on code based assessments, however, we have also experimented with generating feedback for written assessments using the Natural Language Toolkit (NLTK) [12] (which we do not report on further here).

All submitted work should be machine readable/parseable and is intended to be built within some target environment defined by the assessment specification (which is handed out to the students). The educator defines a series of tests which

may then be run, within this specified environment, and all test result are logged and collected. Finally, the logging data is post-processed to produce detailed formative feedback on the student's submitted assessment.

Our cloud based implementation is composed of three layers of code abstractions (which are discussed further in the following subsections):

- 1) layer 1: the boot and config layer - implemented using JClouds[13] and Chef[14]
- 2) layer 2: a monitoring and control layer - implemented using Akka Actors[15]
- 3) layer 3: a workflow and reporting layer - implemented using Apache Camel[16][17].

By implementing our automated assessment solution in this manner, we have found that our feedback software is able to:

- easily adapt to a wide and varied range of assessment requirements
- flexibly incorporate changing assessment requirements
- robustly manage and process varied student submissions (c.f. arbitrary code).

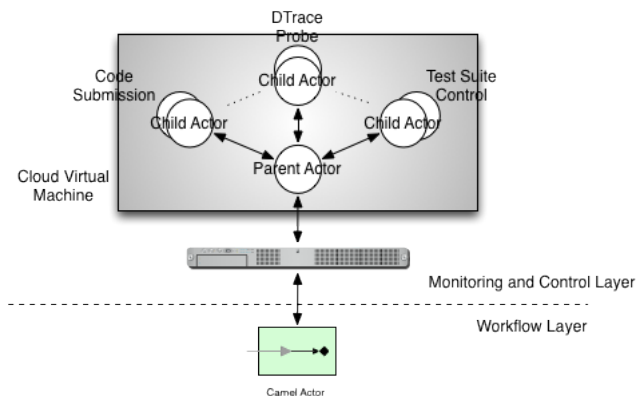


Fig. 4. Actor Based Monitoring and Control

1) Layer 1: Booting and Configuration: As the target build environments can be varied and possibly involve multiple operating systems and machines, we choose to use a virtualised environment. As we need to build, configure, manipulate and destroy these environments multiple times for multiple user submissions, a cloud based solution was selected.

Specifically:

- JClouds[13], via the OpenStack driver, was used to build and control the cloud based virtual machines
- Chef[14] was used to manage the configuration of each virtual machine.

Using the Java compatibility features of Scala[18], the JCloud build and Chef configuration interfaces are controlled within a common programming language.

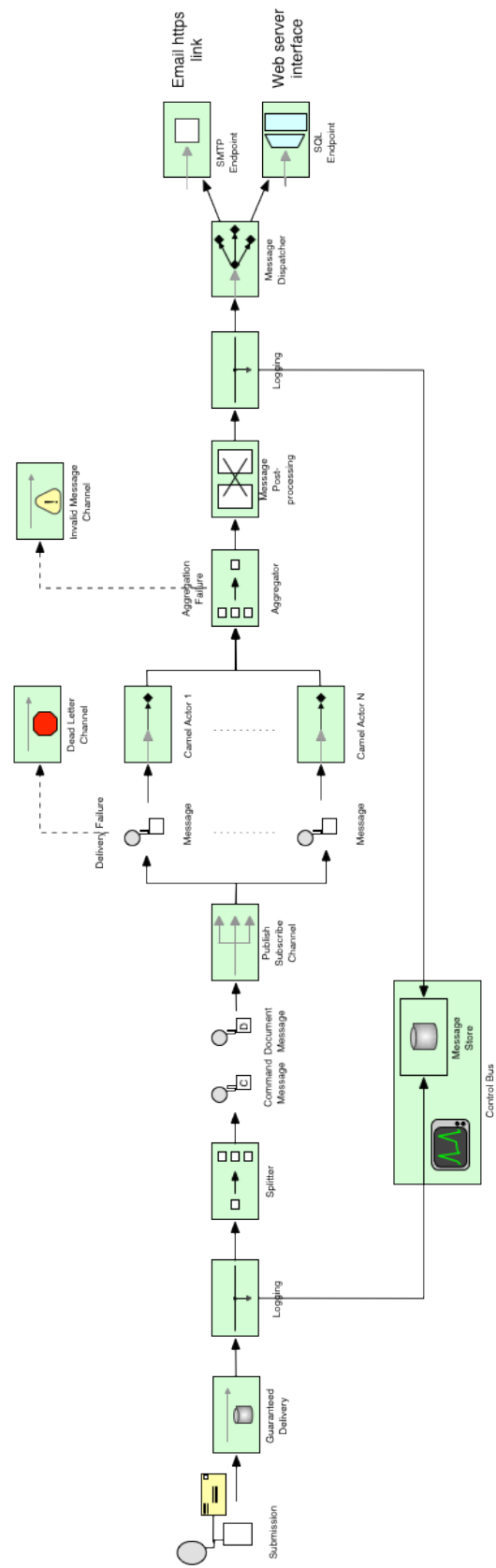


Fig. 5. Generic Workflow

2) *Layer 2: Monitoring and Control:* As student code runs within a virtual machine, it is necessary to control and monitor a range of features. For example:

- test suites may need to be ran against the student code and their logged outputs reported
- RAM, disk, swap, CPU, etc. resource usage may need to be monitored (e.g. via DTrace[19] probes)
- kernel interactions may need to be monitored (e.g. via DTrace[19] probes or system call hooking).

Due to the asynchronous and distributed nature of these control and monitoring components, we encapsulate their implementation using Akka Actors[15] (see figure 4). Actor messaging then allows both the actors to be controlled and for their encapsulated components to communicate information back to a controlling message endpoint.

3) *Layer 3: Workflow and Reporting:* Using Apache Camel[16][17], we are able to define robust and flexible message workflows (see figure 5) that:

- control how logging reports are collected and processed from actors
- how individual messages may be aggregated and transformed
- manage failure handling and recovery
- how feedback reports are communicated back to the student (e.g. via a web server with private links emailed to students).

V. CONCLUSION

The cloud computing paradigm has a great potential to provide a flexible platform for delivering computational resources. This platform has as much merit in a commercial environment as in an academic. The cloud services can help make the experimental sections of the computing course more student centred. Based on the experience from the Workshop on Science Applications and Infrastructure in Clouds and Grids ([20]), acquiring resources through a commercial provision would be too costly for educational purposes, with added privacy, intellectual property and security risks. Therefore institutional private clouds provide a more effective solution.

Using open-source software such as OpenStack on previously purchased research hardware we have implemented an Infrastructure as a Service cloud, and investigated and explored Software as a Service solutions. This feeds into the delivery of our modules such 'Operating Systems' and 'Parallel Computing Architecture'. We have customised cloud deployment to easily create accounts for a large number of students, without compromising the security of our resources.

The *Virtual Laboratory* environment actively promoted student learning and engagement. This also reduced the cost of provisioning hardware, software and lab space, yet delivered an enhanced learning environment. It has significantly improved the student experience and led to a change in the delivery of some modules, to include increased laboratory activity.

Whilst exploring Software as a Service, our automated assessment software has improved the student programming

experience. By providing regular formative feedback, we have been encouraged to set more challenging and demanding assessments.

We anticipate that this approach of deploying Cloud Computing Services would benefit other Schools and H.E. institutions.

REFERENCES

- [1] Mell, Peter and Grance, Tim. (2009) *The nist definition of cloud computing (v15)*. 2009.
- [2] Vaquero, Luis M. Rodero-Merino, Luis. Caceres, Juan and Lindner, Maik. (2008) *A break in the clouds: towards a cloud definition*. SIGCOMM Comput. Commun. Rev., 39(1):50-55, December 2008.
- [3] Garrison, Gary, Kim, Sanghyun and Wakefield, Robin L. (2012) *Success factors for deploying cloud computing*. Commun. ACM, 55(9):6268, September 2012.
- [4] Masud, Anwar Hossain, Yong, Jianming and Huang, Xiaodi. (2012) *Cloud computing for higher education: A roadmap*. In Liang Gao, Weiming Shen, Jean-Paul A. Barth'es, Junzhou Luo, Jianming Yong, Wenfeng Li, and Weidong Li, editors, CSCWD, pages 552557. IEEE, 2012.
- [5] Laszewski et al. (2012) *Comparison of multiple cloud frameworks* Cloud Computing (CLOUD), 2012 IEEE 5th International Conference on. 734-741 2012.
- [6] Vockler et al. (2011) *Experiences using cloud computing for a scientific workflow application* ACM
- [7] LLC OpenStack. 2012. Openstack install and deploy manual.
- [8] Zhang, Jilin. Zhang, Jianhui. Ding, Hong. Wan, Jian. Ren, Yongjian and Wang, Jue. (2013) *Designing and applying an education iaaS system based on openstack*. Appl. Math, 7(1L):155-160, 2013.
- [9] *URANDOM Man Pages*. <http://linux.die.net/man/4/urandom>
- [10] Mircea, Marinela and Andreescu, Anca Ioana (2011). *Using Cloud Computing in Higher Education: A Strategy to Improve Agility in the Current Financial Crisis*, Academy of Economic Studies, Bucharest, Romania, Volume 2011, Article ID 875547, Communications of the IBIMA.
- [11] Thomas, P.Y. (2011), *Cloud computing: A potential paradigm for practising the scholarship of teaching and learning*, Electronic Library, The, Vol. 29 Iss: 2, pp.214 - 224, Emerald Group Publishing Limited
- [12] Bird, Steven, Edward Loper and Ewan Klein (2009). *Natural Language Processing with Python*. O'Reilly Media Inc.
- [13] Cole, Adrian et al. (2009). *JClouds*. Retrieved from: <http://www.jclouds.org/>
- [14] Opscode Inc. *Chef Configuration Management*. Retrieved from: <http://www.opscode.com/chef/>
- [15] Wyatt, Derek (2013). *Akka Concurrency*. Artima Developer.
- [16] Ibsen, Claus and Anstey, Jonathan (2010). *Camel in Action*. Manning Publications Co.
- [17] Hohpe, Gregor and Woolf, Bobby (2003). *Enterprise Integration Patterns*. Addison-Wesley.
- [18] Odersky, Martin, Spoon, Lex, and Venners, Bill (2010). *Programming in Scala*. Artima Developer.
- [19] Gregg, Brendan and Mauro, Jim (2011). *DTrace: Dynamic Tracing in Oracle Solaris, Mac OS X and FreeBSD*. Prentice-Hall.
- [20] Gillam, Lee (2012). *You pay for what you get: Ensuring value for money in the Cloud*. Workshop On Science Applications And Infrastructure In Clouds And Grids. Oxford, UK